成长+趋势量化选股系统（美股/港股）

日更量化选股系统，适配激进成长+趋势动量、高弹性偏好、中线持有的投资风格。系统优先选择趋势强劲、成长性好的高弹性标的，估值容忍度高，追求进攻性收益。

---

1. 项目结构

```
.
├── .env                          # 密钥配置（自动加载，不进git）
├── run_daily.py                  # 主入口（控制台输出 + 参数解析）
├── requirements.txt
├── cache/                        # 当日数据缓存（自动生成，按 ticker 列表 MD5 + 日期命名）
├── outputs/                      # 每日输出（CSV 信号表 / 权重表 / 差异报告 / 持仓历史）
└── quant_system/
    ├── config.py                 # 策略参数、股票池、行业映射、杠杆ETF映射
    ├── data.py                   # 行情 + 基本面数据下载（yfinance + 缓存 + 逐只重试）
    ├── scoring.py                # 三引擎打分（成长/蓝筹/杠杆）+ 线性插值 + 引擎内独立幂次扌
    ├── news.py                   # 新闻抓取（Google News RSS → 全局/个股新闻 → 注入LLM
    ├── llm.py                    # 腾讯混元LLM辅助评分（结合实时新闻，独立判断）
    ├── tracker.py                # 组合收益跟踪（持仓快照 + 区间收益 + 累计收益）
    └── engine.py                 # 主流水线（数据→打分→新闻→LLM融合→选股→权重→跟踪→输出）
```

---

2. 股票池（45只）

池子	引擎	数量	标的
成长池	成长引擎	27	PLTR, NEOV, AAOI, WOLF, COHR, NBIS, CLS, LITE, VRT, GEV, NVDA, AMD, TSLA, MU, CRCL, CLPT, NVTS, SNDK, CNQ, ALM, IPGP, AUR, KLIC, ASX, AMKR, PUMP, 2706.HK
蓝筹池	蓝筹引擎	9	AAPL, MSFT, AMZN, AVGO, ORCL, XLE, B, SCCO, ALB
港股蓝筹	蓝筹引擎	2	0700.HK, 9988.HK
杠杆ETF	杠杆引擎	7	SOXL, DFEN, NVTX, LITX, SNXX, 7747.HK, 7709.HK

> 另外自动加入 3 只参考指数 ETF（SMH、ITA、2800.HK）用于杠杆引擎的指数技术面分析，不参与打分。

---

3. 投资风格：激进高弹性

系统针对激进投资风格做了全面优化：

- 趋势为王：成长引擎趋势权重40%、蓝筹引擎趋势权重30%，强势股获得明显加分
- 容忍高估值：估值权重大幅降至15%，PE/PS 阈值放宽，不因高估值误杀成长股
- 拥抱波动：仓位分配弱化波动率反比惩罚，高弹性股票不被压低仓位
- 放宽风控：成长股单票≤12%、杠杆ETF≤10%、杠杆ETF总计≤25%、单行业≤40%
- 惩罚宽松：估值过热惩罚阈值提高（PE>150才扣分），软惩罚力度降低

---

4. 完整流水线

运行 `python3 run_daily.py [--llm]` 时，依次执行：

```
1. 数据下载 (data.py)

   ├─ yf.download() 批量拉取行情（收盘价 + 成交量）

   ├─ 失败的 ticker 逐只用 yf.Ticker().history() 重试（最多2次）

   ├─ 逐只拉取基本面（PE/PS/ROE/营收增速/现金流等17个字段）

   └─ 当日缓存到 cache/ （按 ticker 列表 MD5 + 日期命名）


2. 技术特征计算 (scoring.py → compute_tech_features)

   ├─ 11个技术指标（详见下文）

   └─ 输出 DataFrame，行=ticker，列=特征


3. 三引擎打分 + 引擎内独立拉伸 (scoring.py)

   ├─ 成长引擎 → score_growth_engine()

   ├─ 蓝筹引擎 → score_bluechip_engine()

   ├─ 杠杆引擎 → score_leverage_engine()

   └─ 每个引擎内部独立幂次拉伸 → merge_scores()


4. 新闻抓取 (news.py, 仅 --llm 启用时)

   ├─ 从 Google News RSS 抓取 6 类全局财经/地缘政治新闻

   ├─ 为每只股票/ETF 抓取相关新闻（普通股按名称、杠杆ETF按行业关键词）

   └─ 当日缓存，避免重复请求


5. LLM 辅助评分 (llm.py, 可选)

   ├─ 将实时新闻注入每只股票的 prompt

   ├─ 对每只股票调用混元大模型
```

```
    └─ 返回 event_score [-1,1] + risk_flag


6. 分数融合 (engine.py → _apply_llm_fusion)

    └─ final_score = 0.8 × quant_score + 0.2 × LLM拉伸分


7. 信号生成 + 权重分配 (engine.py)

    ├─ BUY / HOLD / REDUCE 标记

    ├─ 目标权重计算（含风控约束）

    └─ 与昨日信号对比生成 diff


8. 组合收益跟踪 (tracker.py)

    └─ 持仓快照 → 区间收益 → 累计收益


9. 输出 CSV + 控制台展示
```

---

5. 技术特征计算

`compute_tech_features()` 基于每日收盘价和成交量，计算以下 11 个特征：

特征	计算方式	含义
`ret_5`	`pct_change(5)`	5日涨跌幅
`ret_20`	`pct_change(20)`	20日涨跌幅（月线动量）
`ret_60`	`pct_change(60)`	60日涨跌幅（季线动量）
`above_ma20`	`latest > MA20`	是否站上20日均线（0/1）
`above_ma60`	`latest > MA60`	是否站上60日均线（0/1）

`ma20_slope`	`MA20.pct_change(5)`	MA20近5日变化率（均线斜率）
`breakout_ma60`	昨天低于MA60、今天站上	MA60突破信号（0/1）
`dist_52h`	`latest / 52周高点 - 1`	距52周高点距离（负值=回撤深度）
`drawdown_recovery`	`latest / 20日低点 - 1`	从近20日低点反弹幅度
`vol_20`	`daily_ret.rolling(20).std() × √252`	20日年化波动率
`vol_ratio`	`today_volume / MA20_volume`	量比（放量程度）

---

6. 三引擎评分逻辑

6.1 打分方法：线性插值

所有因子均采用 `_abs_score()` 函数打分——给定一组（阈值，分值）锚点，使用 `np.interp` 在锚点间线性插值，而非传统阶梯函数。这确保分数平滑过渡、最大程度拉开差距。

示例：`ret_20` 的打分锚点为 `[(-0.15, 5), (-0.05, 18), (0.0, 38), (0.03, 58), (0.08, 80), (0.15, 100)]`

- 20日涨幅 -15% → 5分，0% → 38分，+8% → 80分，+15% → 100分
- 中间值如 +5% → 线性插值得 ≈67分

6.2 成长引擎 (`score_growth_engine`)

适用于高增速、趋势动量强的中小至大市值成长股。

大类	权重	子因子及权重
趋势动量	**40%**	ret_5(22%) + ret_20(25%) + ret_60(20%) + MA20斜率(10%) + 站上MA20(8%) + 站上MA60(8%) + MA60突破(7%)
基本面成长	**30%**	营收增速(35%) + 盈利增速(20%) + 毛利率(20%) + 自由现金流(25%)
资金行为	**15%**	量比(35%) + 52周高点距离(35%) + 回撤修复(30%)

估值约束	**15%**	Forward PE(40%) + Trailing PE(35%) + PS(25%)

额外机制：

- 软惩罚（降低力度）：营收增长<0 扣5分、毛利率<0 扣4分、PE>150 扣4分、PS>50 扣3分
- 硬过滤：仅在营收<-30% 且 毛利率<-10% 的极端情况排除（NaN不惩罚，`fillna(0)`）

6.3 蓝筹引擎 (`score_bluechip_engine`)

适用于大市值公司，激进风格下优先选有弹性、高增速的蓝筹。

大类	权重	子因子及权重
趋势	**30%**	ret_20(22%) + ret_60(28%) + 站上MA60(20%) + MA20斜率(15%) + 回撤修复(15%)
成长性	**25%**	营收增速(40%) + 盈利增速(30%) + 毛利率(30%)
质量	**20%**	ROE(30%) + 自由现金流(25%) + 净利率(20%) + 负债率(25%)
估值	**15%**	Trailing PE(35%) + Forward PE(30%) + PS(35%)
韧性	**10%**	回撤修复(65%) + 波动率(35%)

硬过滤：自由现金流<0（NaN通过） 或 负债率>300（NaN通过）排除。

6.4 杠杆ETF引擎 (`score_leverage_engine`)

根据 ETF 是否有底层股票映射或参考指数映射，采用不同权重组合：

类型	示例	**ETF**趋势	底层技术面	底层基本面	参考指数技术面	波动率
有底层股票	NVTX→NVTS, LITX→LITE, SNXX→SNDK	45%	20%	20%	-	15%

有参考指数	SOXL→SMH, DFEN→ITA, 7747/7709→2800	50%	-	-	30%	20%
无参考	（新增无映射的杠杆ETF）	80%	-	-	-	20%

- **ETF**自身趋势：ret_5(30%) + ret_20(30%) + 站上MA20(20%) + MA20斜率(20%)
- 底层技术面：底层股票的 ret_20/ret_60/MA60位置/MA20斜率
- 底层基本面：底层股票的 营收增速/盈利增速/毛利率/Forward PE
- 参考指数技术面：参考指数 ETF 的 ret_5/ret_20/ret_60/MA60位置/MA20斜率
- 波动率：年化波动率 0.2→90分, 0.8→28分, 1.0→10分（激进风格，高波仅轻度惩罚）

---

7. 引擎内独立幂次拉伸（对抗分数收敛）

多因子加权平均天然会让分数向中间段收敛（大部分股票集中在50~70分），不利于决策。

7.1 问题：引擎间系统性偏差

三个引擎的原始分数分布有显著差异：

- 蓝筹引擎：因子值普遍较好（高ROE、正现金流、低波动），原始分多在 60~85 区间
- 成长引擎：因子值波动大（有的爆发增长、有的亏损），原始分分布广，15~70 都有
- 杠杆引擎：波动率高，原始分偏低

如果在三引擎合并后统一做幂次拉伸，蓝筹的原始高分会被进一步放大，成长股即使引擎内排名第一也可能不如蓝筹的中等水平，导致组合中蓝筹系统性占优。

7.2 解决方案：引擎内独立拉伸

系统在 `merge_scores()` 中，先对每个引擎的 **`quant_score`** 独立做幂次拉伸，再合并：

```
# merge_scores() 中的关键逻辑

stretched_dfs = []
```

```
for df in [growth_df, bluechip_df, leverage_df]:
    d = df.copy()
    d["quant_score_raw"] = d["quant_score"].copy()        # 保留原始分用于诊断
    d["quant_score"] = _stretch_scores(d["quant_score"], power=1.8)  # 引擎内独立拉伸
    stretched_dfs.append(d)
merged = pd.concat(stretched_dfs, axis=0)                 # 合并三引擎
```

7.3 `_stretch_scores` 拉伸函数

```
步骤：
1. 归一化到 [0, 1]：normed = (score - min) / (max - min)   ← 在本引擎内归一化
2. 幂次变换：stretched = normed ^ 1.8
3. 映射回 [0, 100]：final = stretched × 100
```

7.4 效果

- 每个引擎的第一名都能拿到接近 **100** 的高分，不受其他引擎分布影响
- 成长引擎内的强势股和蓝筹引擎内的强势股站在同一起跑线
- power > 1 使得高分被拉高、低分被压低，分数分布从密集区展开
- 原始分数保留在 `quant_score_raw` 列用于诊断

---

8. 新闻抓取模块

`news.py` 从海外主流媒体自动抓取实时新闻，注入 LLM prompt，让大模型基于真实时事而非训练数据做出判断。

8.1 数据源

- **Google News RSS**（免费，无需 API Key）
- 聚合 Reuters、Bloomberg、CNBC、BBC、AP 等主流媒体
- 按关键词搜索，返回最新新闻标题

8.2 全局新闻

系统抓取 6 类全球宏观/地缘政治关键词的最新新闻：

关键词	覆盖方向
`stock market today`	全球股市动态
`geopolitics war conflict`	地缘政治/战争冲突
`Federal Reserve interest rate`	美联储利率政策
`US China trade tariff`	中美贸易/关税
`oil price OPEC`	油价/OPEC
`semiconductor chip AI`	半导体/AI行业

去重后保留最多 30 条全局新闻标题。

8.3 个股新闻

针对每只股票/ETF 分别抓取相关新闻：

- 普通股票：使用 `{公司名} stock` 搜索（如 `Palantir Technologies stock`）
- 杠杆**ETF**：根据行业描述匹配关键词搜索：
- 半导体类 → `semiconductor chip industry news`
- 航空国防类 → `defense military geopolitics news`
- 纳指类 → `Nasdaq 100 tech stocks news`
- 恒指类 → `Hong Kong stock market news`
- 每只标的保留最多 6 条新闻标题

8.4 缓存策略

- 当日缓存到 `cache/news_cache.pkl`
- 同一天多次运行只抓取一次，避免触发频率限制
- 次日自动失效，确保新闻时效性

8.5 Prompt 注入格式

```
【近期全球财经/地缘政治要闻】

1. Fed holds interest rates steady amid tariff uncertainty

2. Israel strikes Iran nuclear facilities, oil prices surge

...



【SOXL 相关新闻】

1. Semiconductor stocks rally on AI demand outlook

2. TSMC reports record revenue as chip demand surges

...
```

---

9. LLM辅助评分

可选启用腾讯混元大模型（`--llm`），对每只股票独立生成事件评分。

9.1 核心原则：正交互补

- **LLM** 不接收任何量化分数，完全基于公司背景、实时新闻、宏观环境独立判断
- 量化引擎擅长「价量趋势 + 财务数据」，LLM 擅长「新闻事件 + 宏观研判」，两者正交互补

- 新闻注入：系统自动抓取最新新闻标题注入 prompt，解决大模型训练数据时效性不足的问题

9.2 Prompt 设计

个股 **prompt**（传入公司信息 + 实时新闻，不传量化分数）：

- 股票代码、名称、行业、市值、营收增速、毛利率、年化波动率
- 全局财经/地缘政治新闻标题（最多10条）
- 该股票相关新闻标题（最多5条）
- 要求结合真实新闻分析：行业影响、地缘政治、宏观环境、公司事件
- 输出：`{"event_score": -1~1, "risk_flag": 0/1, "reason": "..."}`

杠杆**ETF** 定制 **prompt**（传入 ETF 行业描述 + 实时新闻）：

- ETF 代码、产品描述（如 "3倍做多半导体指数ETF，受AI芯片需求、中美科技竞争影响"）
- 全局及行业相关新闻标题
- 要求结合真实新闻分析：地缘政治、宏观政策、行业周期、突发事件
- 特别关注杠杆ETF特有风险（波动率衰减、持仓成本）

9.3 融合公式

```
raw_llm ∈ [-1, 1] → 归一化 [0, 1] → 幂次拉伸 (power=2.0) → 映射 [0, 100]



final_score = 0.8 × quant_score + 0.2 × llm_stretched

若 risk_flag = 1，额外扣 10 分
```

LLM 的 event_score 也做了幂次拉伸（power=2.0），放大正面/负面评价之间的差距，避免 LLM 分数集中在小范围内对排序无影响。

9.4 调用方式

支持两种调用方式（自动选择）：

1. 腾讯云 **SDK**：配置 `TENCENT_SECRET_ID` + `TENCENT_SECRET_KEY`
2. **OpenAI** 兼容 **API**：配置 `HUNYUAN_API_KEY`

JSON 解析兼容 thinking 模型的 `...` 输出格式。

---

10. 选股与权重分配

10.1 信号生成（`engine.py`）

按 `final_score` 从高到低排序后：

- **BUY**：排名前 `top_n_buy=12` 且 `final_score ≥ 55`
- **HOLD**：其余通过硬过滤且 `final_score ≥ 45`
- **REDUCE**：硬过滤不通过 或 `final_score < 45`

10.2 权重分配（`_build_target_weights`）

```
raw = max(final_score - 40, 0)                # 超额分（基准线40）

vol_adj = clip(annual_vol, 0.15, 0.80)        # 波动率裁剪

raw = raw / (0.5 + 0.5 × vol_adj)             # 弱化波动率惩罚

weight = raw / sum(raw)                        # 归一化
```

弱化波动率：使用 `score / (0.5 + 0.5×vol)` 而非传统 `score / vol`，确保高弹性股票不被过度压低仓位。

10.3 风控约束

依次执行以下约束（超限则等比缩放）：

约束	上限
成长股单票	≤ 12%
蓝筹股单票	≤ 15%

杠杆ETF单票	≤ 10%
杠杆ETF总计	≤ 25%
单行业总计	≤ 40%

最后再做一次归一化确保权重之和 = 100%。

---

11. 组合收益跟踪

`tracker.py` 内置组合收益跟踪模块，持续监控目标仓位表现。

工作流程

```
每次运行：

1. 读取 portfolio_history.csv 中上一期持仓快照

2. 用当前最新价格计算上一期每个持仓的区间收益

3. 加权汇总得到本期组合收益

4. 累计收益 = (1 + 上期累计) × (1 + 本期收益) − 1

5. 记录本期新持仓快照（ticker / 权重 / 入场价格）

6. 保存到 portfolio_history.csv
```

持久化字段

字段	说明
`snapshot_date`	快照日期
`ticker`	标的代码

`weight`	目标权重
`entry_price`	入场价格（当日最新收盘价）
`period_return`	本期组合加权收益率
`cumulative_return`	从首次跟踪开始的累计收益率

换仓处理

目标仓位刷新时，自动以新权重 + 新入场价格记录新快照，旧持仓的收益已在本次计算中结算。

---

12. 数据来源与缓存

所有行情和基本面数据来自 **Yahoo Finance**（免费，无需API Key）。新闻来自 **Google News RSS**（免费，无需API Key）。

行情数据

- 批量调用 `yf.download()` 拉取收盘价 + 成交量（约320个交易日）
- 批量下载失败的 ticker 会逐只用 `yf.Ticker().history()` 重试（最多2次，间隔1秒）
- 重试时自动处理时区（`tz_localize(None)`），确保与批量下载的 tz-naive 索引兼容

基本面数据

- 逐只调用 `yf.Ticker().info` 拉取 17 个字段：
- 市值、PE(trailing/forward)、PS、PB
- 毛利率、净利率、营收增速、盈利增速
- ROE、自由现金流、经营现金流、负债率
- Beta、平均成交量、行业、简称

新闻数据

- 从 Google News RSS 抓取（聚合 Reuters/Bloomberg/CNBC/BBC 等主流媒体）
- 6 类全局关键词 + 每只标的个性化搜索
- 当日缓存到 `cache/news_cache.pkl`，次日自动失效

缓存策略

- 行情/基本面缓存路径：`cache/{label}_{md5(tickers)[:10]}_{YYYYMMDD}.pkl`
- 新闻缓存路径：`cache/news_cache.pkl`
- 同一天同一 ticker 列表只下载一次，重跑直接读缓存
- 股票池变更后 MD5 变化，自动触发重新下载
- 可手动删除 `cache/*.pkl` 强制刷新

数据过滤

- 价格数据不足 `min_price_days=60` 天的标的被过滤（日志会输出被过滤的列表）
- 参考指数 ETF（SMH/ITA/2800.HK）仅用于杠杆引擎，不进入打分

---

13. 安装与运行

```
cd /path/to/project

python3 -m pip install -r requirements.txt



# 纯量化（不调用 LLM）

python3 run_daily.py



# 启用混元LLM辅助评分（含新闻抓取）

python3 run_daily.py --llm



# 指定输出目录

python3 run_daily.py --out my_outputs
```

编辑 `.env` 配置LLM密钥（仅 `--llm` 功能需要）：

免责声明

本项目仅用于研究与学习，不构成任何投资建议。